

Cây chỉ số và truy vấn đoạn (Fenwick, Segment Trees, Range Queries)

PHỤ THUỘC B/06 cho tổng tiền tố — chương này là câu trả lời khi mảng thay đổi; B/09 cho ý tưởng tăng cường; A/05 cho tính kết hợp.

TÓM TẮT

Chương B/06 cho truy vấn đoạn $O(1)$ với điều kiện mảng đứng yên. Chương này bỏ điều kiện đó, và cái giá phải trả là $O(\log n)$ cho cả truy vấn lẫn cập nhật. Bốn cấu trúc: Fenwick — gọn cho tổng/XOR với cập nhật điểm; cây phân đoạn — tổng quát nhất, thêm lazy propagation thì cập nhật cả đoạn cũng $O(\log n)$; sparse table — $O(1)$ truy vấn nhưng mảng phải tĩnh; và phân rã căn bậc hai — chậm hơn nhưng dùng được cho những phép toán mà ba cấu trúc kia chịu. Các cấu trúc gộp đoạn cần trạng thái đủ thông tin và phép gộp kết hợp; Mo cần thao tác thêm/bớt ở biên đủ rẻ. Nếu chỉ nhớ một điều: chọn cấu trúc bằng cách trả lời hai câu — phép toán có kết hợp không, và mảng có thay đổi không.

KIẾN THỨC CẦN CÓ TRƯỚC

- **phép toán kết hợp (associative)** — phép $\oplus$ thoả $(a \oplus b) \oplus c = a \oplus (b \oplus c)$. Điều kiện để gộp kết quả của hai nửa thành kết quả của cả đoạn — nền của mọi cấu trúc trong chương.
- **phần tử trung hoà** — giá trị e với $a \oplus e = e \oplus a = a$; cần để trả lời truy vấn trên đoạn rỗng mà không phải viết trường hợp riêng. Cùng với phép kết hợp, ta có một *vị nhóm* (monoid).
- **Fenwick (binary indexed tree)** — mảng phụ trong đó ô i giữ tổng của một đoạn có độ dài bằng bit thấp nhất của i ; cập nhật điểm và tổng tiền tố đều $O(\log n)$, code chỉ vài dòng.
- **cây phân đoạn (segment tree)** — cây nhị phân trong đó mỗi nút giữ kết quả gộp của một đoạn; lá là phần tử đơn, gốc là cả mảng.
- **lazy propagation** — kỹ thuật hoãn việc đẩy cập nhật xuống các nút con cho tới khi thật sự cần, để cập nhật cả đoạn cũng chỉ $O(\log n)$.
- **sparse table** — bảng tiền xử lý $f[i][k] =$ kết quả gộp của đoạn dài 2^k bắt đầu tại i ; truy vấn $O(1)$ cho phép toán *luỹ đẳng* như cực trị, nhưng mảng phải tĩnh.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao Fenwick không dùng được cho truy vấn cực đại, trong khi cây phân đoạn thì được?
- Q2. Sparse table trả lời cực đại trong $O(1)$ còn cây phân đoạn cần $O(\log n)$. Vì sao sparse table không thay thế được cây phân đoạn?
- Q3. Lazy propagation hoãn việc cập nhật lại. Vì sao việc hoãn đó không làm sai kết quả, và lập luận nào bảo đảm điều đó?
- Q4. Cùng $O(\log n)$ nhưng Fenwick thường nhanh hơn cây phân đoạn vài lần. Khác biệt nằm ở đâu?
- Q5. Có phép toán nào mà cả Fenwick lẫn cây phân đoạn đều chịu, nhưng phân rã căn bậc hai vẫn giải được không?

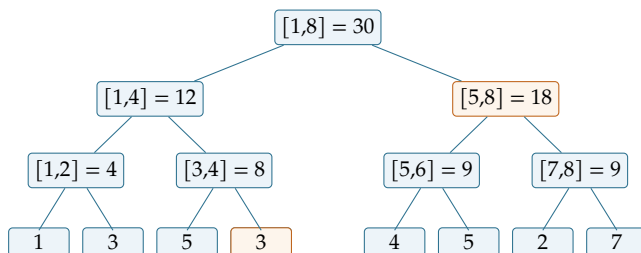
Cuối chương B/06 còn một câu hỏi bỏ ngỏ. Tổng tiền tố cho truy vấn đoạn trong $O(1)$, nhưng chỉ khi mảng đứng yên; mảng hiệu cho cập nhật đoạn trong $O(1)$, nhưng chỉ khi ta đọc kết quả một lần ở cuối. Bài toán thật thường trộn cả hai: cập nhật và truy vấn xen kẽ, mỗi loại hàng trăm nghìn lần.

Không có biểu diễn tĩnh nào làm được cả hai, và lý do khá sâu: tổng tiền tố nhanh vì mỗi ô của nó tóm tắt *rất nhiều* phần tử, nên sửa một phần tử làm hỏng rất nhiều ô. Đây là một đánh đổi thật sự, không phải thiếu sót của kỹ thuật. Lời giải của chương này là một sự thỏa hiệp có cấu trúc: thay vì mỗi ô tóm tắt một tiền tố dài, hãy để mỗi ô tóm tắt một đoạn có độ dài *lưu thừa của 2*, xếp thành cây. Khi đó mỗi phần tử chỉ nằm trong $O(\log n)$ đoạn, và mỗi đoạn truy vấn ghép được từ $O(\log n)$ mảnh.

1.1 Cây phân đoạn: ý tưởng tổng quát nhất

Chia mảng làm đôi, mỗi nửa lại chia đôi, cho tới khi còn một phần tử. Mỗi nút của cây giữ kết quả gộp của đoạn tương ứng. Với tổng, nút cha bằng tổng hai con; với cực đại, nút cha bằng cực đại hai con.

Truy vấn đoạn $[l, r]$: đi từ gốc xuống, nếu đoạn của nút nằm gọn trong $[l, r]$ thì trả kết quả đã lưu, nếu không giao thì bỏ qua, nếu giao một phần thì đi tiếp cả hai con. Có thể chứng minh mỗi tầng đóng góp tối đa hai nút được “dùng nguyên”, nên chi phí là $O(\log n)$. Cập nhật một phần tử: đi từ lá lên gốc, cập nhật $O(\log n)$ nút trên đường.



Truy vấn $[4, 8]$ ghép từ hai nút đã tô: $[4, 4]$ và $[5, 8]$, tổng $3 + 18 = 21$. Mỗi tầng gộp tối đa hai nút $\Rightarrow O(\log n)$.

Hình 1.1. Cây phân đoạn cho mảng tám phần tử. Điều đáng chú ý: truy vấn không đi tới lá mà dừng lại ngay khi gặp một nút nằm gọn trong đoạn hỏi. Đây cũng là chỗ giải thích vì sao lazy propagation hoạt động — nếu ta dừng ở một nút thì mọi thứ bên dưới nó không cần chính xác ngay lúc này.

Điều kiện để mọi thứ trên đúng là *tính kết hợp*: kết quả của cả đoạn phải ghép được từ kết quả hai nửa. Tổng, tích, cực đại, cực tiểu, GCD, XOR, số lượng, “đoạn con có tổng lớn nhất” đều thỏa. Ngược lại, “phần tử xuất hiện nhiều nhất trong đoạn” thì *không* — biết đáp án của hai nửa không đủ để suy ra đáp án của cả đoạn — và đó là lý do bài đó cần công cụ khác, ví dụ thuật toán Mo ở mục 5.

Nhận xét này là cùng một điều kiện với “thông tin tăng cường phải tổng hợp được từ hai con” ở chương B/09, nay được gọi bằng tên toán học của nó. Trong ngôn ngữ đại số, ta cần một *vị nhóm*: một phép kết hợp cộng một phần tử trung hoà. Phần tử trung hoà không phải chi tiết hình thức — nó chính là giá trị bạn trả về cho đoạn rỗng, và quên nó là nguồn lỗi cài đặt phổ biến (trả về 0 cho truy vấn cực đại là sai nếu mảng có số âm; phải trả về $-\infty$).

Lazy propagation: hoãn cho tới khi thật cần

Cập nhật một phần tử là $O(\log n)$, nhưng cập nhật cả một đoạn — “cộng 5 vào mọi phần tử trong $[l, r]$ ” — nếu làm từng phần tử thì thành $O(n \log n)$. Lazy propagation giải quyết bằng một ý rất giống ý của mảng hiệu: *ghi nợ*.

Khi một cập nhật phủ trọn đoạn của một nút, ta cập nhật giá trị tổng hợp của nút đó và ghi một “khoản nợ” vào nút, rồi *dừng lại* — không đi xuống các con. Chỉ khi nào một truy vấn hay cập nhật sau này cần đi xuống dưới nút đó, ta mới đẩy khoản nợ xuống hai con.

Vì sao không sai? Vì bất biến được duy trì là: *giá trị lưu tại mỗi nút đã đúng, còn khoản nợ tại nút mô tả phần chưa được áp cho các con*. Mọi đường đi xuống đều đẩy nợ trước khi đọc con, nên không ai đọc dữ liệu cũ. Đây lại là một ví dụ của mẫu “trì hoãn công việc cho tới khi có người thật sự cần” — cùng ý với xoá lười ở chương B/07.

1.2 Fenwick: ngắn hơn, nhanh hơn, hẹp hơn

Fenwick làm cùng việc với cây phân đoạn cho trường hợp tổng, nhưng bằng một mẹo chỉ số rất gọn: ô i giữ tổng của đoạn kết thúc tại i có độ dài bằng *bit thấp nhất* của i . Nhờ vậy tổng tiền tố tới i ghép từ các ô i , $i - \text{lowbit}(i)$, $\dots$ — số ô bằng số bit 1 của i , tối đa $\log n$.

Ưu điểm là rất thực dụng: code khoảng năm dòng, bộ nhớ đúng n số, và thường có hằng số nhỏ. Các bước truy cập nhảy theo lowbit, không phải quét tuần tự; mức nhanh hơn phải đo trên cách cài và dữ liệu cụ thể.

Cái giá của khuôn Fenwick tổng/XOR: truy vấn đoạn dùng nghịch đảo để loại tiền tố trái, còn cập nhật điểm kiểu cộng dồn dùng tính giao hoán. Không nên suy rộng rằng mọi biến thể Fenwick đều cần nghịch đảo: cực tiểu tiền tố với cập nhật chỉ giảm vẫn làm được, nhưng không hỗ trợ gán tùy ý hay suy ra cực tiểu đoạn bằng hai tiền tố. Tổng và XOR có; cực đại thì không — biết cực đại của $[1, r]$ và của $[1, l - 1]$ không suy ra được cực đại của $[l, r]$. Đó là câu trả lời cho câu hỏi mở đầu thứ nhất, và cũng là ranh giới rõ ràng giữa hai cấu trúc.

Hai mở rộng đáng biết: Fenwick cũng làm được “cập nhật đoạn, truy vấn điểm” bằng cách chạy trên mảng hiệu; và làm được cả “cập nhật đoạn, truy vấn đoạn” bằng hai Fenwick với một chút đại số. Ngoài ra, có thể tìm “vị trí nhỏ nhất mà tổng tiền tố vượt x ” trong $O(\log n)$ bằng cách đi xuống theo bit, với điều kiện các phần tử không âm để tổng tiền tố đơn điệu — kỹ thuật thay thế cho cây thống kê thứ tự ở B/09 và thường nhanh hơn nhiều.

1.3 Sparse table: trả trước tất cả, với điều kiện mảng tĩnh

Nếu mảng không bao giờ thay đổi và phép toán *luỹ đẳng* — tức $a \oplus a = a$, đúng với cực đại, cực tiểu, GCD — thì ta có thể làm tốt hơn $O(\log n)$. Tiền xử lý $f[i][k] =$ kết quả gộp của đoạn dài 2^k bắt đầu tại i , tốn $O(n \log n)$ thời gian và bộ nhớ. Truy vấn $[l, r]$: lấy $k = \lfloor \log_2(r - l + 1) \rfloor$ và gộp hai đoạn dài 2^k , một bắt đầu tại l , một kết thúc tại r . Hai đoạn này *chồng lên nhau*, và đó chính là chỗ cần tính lũy đẳng — với tổng thì cách này sai vì phần chồng bị đếm hai lần.

Vậy nên bảng so sánh dưới đây tóm tắt lựa chọn, và nó đáng thuộc vì đây là quyết định bạn phải ra rất nhiều lần.

Cấu trúc	Truy vấn	Cập nhật	Điều kiện	Chọn khi
Tổng tiền tố (B/06)	$O(1)$	$O(n)$	có nghịch đảo	Mảng tĩnh, phép có nghịch đảo
Sparse table	$O(1)$	$O(n)$ sửa ô bị ảnh hưởng	kết hợp, lũy đẳng	Mảng tĩnh, hỏi cực trị rất nhiều
Fenwick	$O(\log n)$	$O(\log n)$	tổng/XOR, cộng dồn	Tổng có cập nhật; ưu tiên vì ngắn và nhanh
Cây phân đoạn	$O(\log n)$	$O(\log n)$	kết hợp	Phép toán bất kỳ có kết hợp; cần cập nhật đoạn (lazy)
Phân rã $\sqrt{n}$	$O(\sqrt{n})$	$O(1)$ hoặc $O(\sqrt{n})$	gộp khối đủ rẻ	Tổng/cực trị theo khối; kiểm tra chi phí gộp

1.4 Phân rã căn bậc hai và thuật toán Mo

Khi không tìm được trạng thái gọn để gộp hai đoạn, hãy xét xử lý theo khối; đây không phải lời giải tự động cho mọi phép toán không kết hợp. Lúc đó vẫn còn một hướng: chia mảng thành $\sqrt{n}$ khối, mỗi khối $\sqrt{n}$ phần tử, và giữ thông tin tổng hợp cho từng khối. Truy vấn đoạn xử lý các khối nằm trọn bằng thông tin đã lưu, và duyệt tay các phần tử ở hai đầu — tổng cộng $O(\sqrt{n})$ nếu mỗi khối được gộp trong $O(1)$.

Cần thiết kế và phân tích thao tác gộp/cập nhật khối; đổi lại cấu trúc khá linh hoạt, và code thường ngắn hơn cây phân đoạn có lazy. Trong nhiều bài, $O(n\sqrt{n})$ vẫn nằm trong ngân sách của chương A/03 và đó là lựa chọn hợp lý — đặc biệt khi bạn còn ít thời gian trong kỳ thi.

Một biến thể rất đẹp là *thuật toán Mo*: khi mọi truy vấn được biết trước (truy vấn ngoại tuyến), hãy sắp xếp chúng theo khối của đầu trái rồi theo đầu phải, và di chuyển hai con trỏ dần dần từ truy vấn này sang truy vấn kế tiếp. Tổng quãng đường của hai con trỏ là $O((n+q)\sqrt{n})$, và ta giải được những bài mà cây phân đoạn không đựng tới được — ví dụ “có bao nhiêu giá trị phân biệt trong đoạn” hay “phần tử xuất hiện nhiều nhất trong đoạn”. Chú ý điều kiện đánh đổi: kỹ thuật này đòi truy vấn phải biết trước, tức là bạn mua tốc độ bằng cách từ bỏ tính trực tuyến.

1.5 Ba mở rộng nên biết là có

Cây phân đoạn bền vững. Mỗi cập nhật chỉ đụng $O(\log n)$ nút, nên thay vì sửa tại chỗ, ta tạo bản sao của đúng đường đó và chia sẻ phần còn lại với phiên bản cũ. Kết quả: giữ được mọi phiên bản với chi phí $O(\log n)$ mỗi lần. Ứng dụng kinh điển là truy vấn “phần tử nhỏ thứ k trong đoạn $[l, r]$ ” — giải bằng hiệu của hai phiên bản.

Hai chiều. Cây phân đoạn của các cây phân đoạn cho truy vấn trên hình chữ nhật với $O(\log^2 n)$. Tổn bộ nhớ, và trong thực tế thường thay bằng Fenwick hai chiều nếu chỉ cần tổng.

Cây phân đoạn “beats”. Xử lý được các cập nhật lạ như “gán mọi phần tử trong đoạn bằng $\min(a_i, x)$ ” với phân tích khấu hao tinh vi. Xếp vào loại biết là có, tra khi gặp.

CẠM BẮY

Lỗi cài đặt phổ biến nhất với cây phân đoạn không nằm ở logic cây mà ở *phần tử trung hoà*. Truy vấn cực đại trên đoạn rỗng phải trả về $-\infty$, không phải 0; truy vấn GCD trên đoạn rỗng phải trả về 0, không phải 1. Lỗi này ẩn kỹ vì nó chỉ lộ ra khi mảng có số âm hoặc khi truy vấn rơi vào nhánh không giao. Cách phòng: viết rõ phần tử trung hoà thành một hằng số có tên, và tự hỏi “nếu đoạn rỗng thì hàm này trả về gì”.

1.6 Biến cấu trúc tĩnh thành cấu trúc động: phép biến đổi Bentley–Saxe

Sparse table ở mục 3 trả lời truy vấn trong $O(1)$ nhưng đòi hỏi mảng đứng yên. Đó không phải trường hợp cá biệt: rất nhiều cấu trúc mạnh đều là cấu trúc *tĩnh* — mảng đã sắp cộng tìm kiếm nhị phân, bao lồi dựng sẵn, cây phân hoạch không gian. Câu hỏi tự nhiên là có cách chung nào thêm phép chèn vào một cấu trúc tĩnh bất kỳ không, thay vì phải thiết kế lại từ đầu cho từng bài. Bentley và Saxe[1] trả lời là có, với một điều kiện rõ ràng, và lời giải của họ đơn giản tới mức nên thuộc.

Điều kiện: bài toán truy vấn phải *phân rã được*, nghĩa là khi tập dữ liệu tách thành hai phần rời nhau thì đáp án trên cả tập gộp được từ hai đáp án thành phần bằng một phép toán rẻ. Cụ thể, cực tiểu, cực đại, tổng, “có tồn tại phần tử thoả tính chất này không”, “điểm gần nhất tới q ” đều phân rã được. “Phần tử nhỏ thứ k ” thì không — biết phần tử thứ k của mỗi nửa không cho biết phần tử thứ k của cả tập.

Cách dựng đọc như một phép đếm nhị phân, và đó chính là trực giác nên giữ. Ta không nuôi một cấu trúc lớn mà nuôi một *bộ* cấu trúc tĩnh có kích thước 1, 2, 4, 8, ..., mỗi luỹ thừa của hai có nhiều nhất một cái — đúng như biểu diễn nhị phân của số phần tử hiện có. Chèn một phần tử là dựng một cấu trúc kích thước 1; để có hai cấu trúc cùng kích thước thì huỷ cả hai và dựng lại một cấu trúc tĩnh kích thước gấp đôi, đúng như việc nhớ 1 khi cộng nhị phân. Truy vấn thì hỏi cả $O(\log n)$ cấu trúc rồi gộp các đáp án lại.

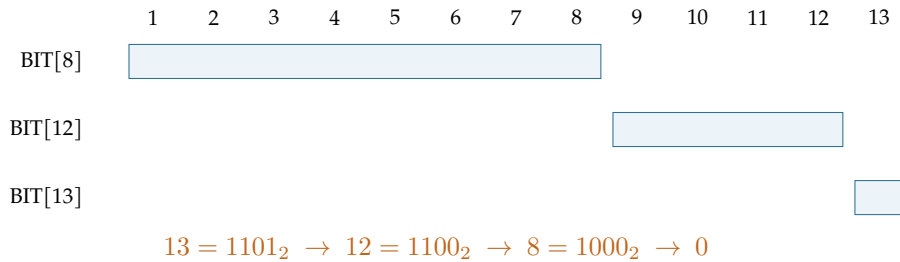
Chi phí suy ra trực tiếp từ hình ảnh đó. Mỗi phần tử chỉ bị dựng lại một lần cho mỗi tầng nó leo qua, tức là $O(\log n)$ lần trong cả đời; nên nếu dựng cấu trúc tĩnh cho n phần tử tốn $P(n)$ thì chèn tốn $O(\frac{P(n)}{n} \log n)$ khấu hao, còn truy vấn tốn thêm một thừa số $O(\log n)$ so với bản tĩnh. Lập luận này chính là lập luận bộ đếm nhị phân ở chương A/03, xuất hiện lại ở một vai trò hoàn toàn khác.

Ba hệ quả thực dụng. Một, khi bạn có một cấu trúc tĩnh viết sẵn và bài lại đòi thêm phần tử *trực tuyến*, đừng vội đi tìm cấu trúc động tương ứng — hãy hỏi bài toán có phân rã được không đã. Hai, phép biến đổi này không cho phép xoá; muốn xoá phải dùng các biến thể khác của cùng dòng nghiên cứu, và trong bài thi thì thường rẻ hơn nhiều nếu chuyển sang xử lý ngoại tuyến bằng chia để trị trên trục thời gian ($C/14$). Ba, nếu đã có một cấu trúc động tự nhiên — cây phân đoạn cho các phép toán kết hợp là ví dụ ngay trong chương này — thì dùng nó, vì phép biến đổi tổng quát luôn phải trả thêm một thừa số lôgarit cho tính tổng quát.

Hình dạng “nhiều tầng, mỗi tầng gấp đôi, hợp nhất khi đầy, truy vấn phải hỏi mọi tầng” còn xuất hiện ở nơi khác hẳn: các động cơ lưu trữ ghi nhiều đọc ít trong cơ sở dữ liệu hiện đại tổ chức dữ liệu đúng theo kiểu đó. Sự trùng hình dạng ấy không phải quan hệ vay mượn có chứng cứ lịch sử, nhưng nó cho thấy cùng một ràng buộc — ghi phải rẻ, và cấu trúc tĩnh thì rẻ hơn cấu trúc động — dẫn tới cùng một lời giải ở hai ngành cách nhau ba chục năm.

1.7 Nhìn lowbit và kiểm hợp thành lazy bằng tay

Hình 1.2 triển khai trực tiếp truy vấn tiền tố 13. Mỗi bước xoá bit 1 thấp nhất nên các khối vừa khít, không chồng lấn. Để tự kiểm cài đặt, thử thêm Δ tại vị trí 11: với $n = 16$, các ô bị cập nhật phải là 11, 12, 16.



Hình 1.2. Tiền tố $[1, 13]$ là hợp rời của $[13, 13]$, $[9, 12]$, $[1, 8]$. Vì thế tổng cần đúng ba ô Fenwick: $\text{BIT}[13] + \text{BIT}[12] + \text{BIT}[8]$.

Lazy propagation còn đòi cập nhật tác động được lên trạng thái tổng hợp và hợp thành được trong thời gian hằng số[2]. Với cộng đoạn và hỏi tổng, lưu (sum, len) ; cộng d biến nó thành $(sum + d \cdot len, len)$. Không lưu độ dài thì không biết phải tăng tổng bao nhiêu. Với cả gán và cộng, thứ tự là phần của thuật toán: gán 5 rồi cộng 3 cho kết quả 8, còn cộng 3 rồi gán 5 cho kết quả 5. Hãy kiểm hai chuỗi thao tác này trên mảng hai phần tử trước khi thử ngẫu nhiên.

CẠM BẤY

Không phải mọi phép gộp kết hợp đều tự động hỗ trợ mọi cập nhật lazy. Ví dụ chỉ lưu tổng thì không đủ cập nhật $a_i \leftarrow \min(a_i, x)$: hai đoạn $[0, 10]$ và $[5, 5]$ cùng tổng 10, nhưng chặn trên ở 5 cho tổng 5 và 10. Muốn xử lý phải thêm thông tin và chứng minh chi phí, như segment tree beats.

1.8 Phản biện, nhược điểm và rủi ro triển khai

Cây phân đoạn và Fenwick thường tạo ra ảo tưởng rằng mọi bài toán truy vấn đoạn đều có thể giải quyết nhanh chóng bằng cách gõ một khung cây quen thuộc. Tuy nhiên, cái giá về tài nguyên và các cạm bẫy logic khi cài đặt lazy propagation là những rào cản rất thực tế.

Phản biện: sự lạm dụng cây phân đoạn (Segment Tree Overkill). Cây phân đoạn là cấu trúc bị lạm dụng nhiều nhất trong lập trình thi đấu. Khi thấy truy vấn đoạn, người mới thường lập tức gõ cây phân đoạn dài cả trăm dòng mã. Nhưng nếu dữ liệu tĩnh, Sparse Table cho phép trả lời RMQ trong thời gian $O(1)$ tuyệt đối — nhanh hơn $O(\log n)$ từ 5 tới 10 lần. Nếu bài toán chỉ cần cập nhật điểm và truy vấn tổng/ngịch đảo, cây Fenwick viết chỉ trong 10 dòng, chạy nhanh gấp 3–5 lần nhờ tận dụng hoàn hảo cache L1 và chỉ tốn đúng $1N$ bộ nhớ thay vì $4N$. Lạm dụng cây phân đoạn không chỉ làm code cồng kềnh, khó gỡ lỗi mà còn khiến hệ số hằng số thời gian phình to, dễ dẫn tới lỗi quá thời gian (TLE) một cách đáng tiếc.

Nhược điểm cốt lõi về tài nguyên và giới hạn cấu trúc:

- **Gánh nặng bộ nhớ $4N$ và phân mảnh heap:** Cây phân đoạn dạng mảng cần mảng kích thước tối thiểu $4N$. Nếu mỗi nút phải lưu các cấu trúc phức tạp (như mảng vector hay ma trận kích thước nhỏ), dung lượng RAM sẽ bùng nổ hàng trăm megabyte. Nếu cài đặt bằng con trỏ động (new Node), chi phí lưu con trỏ 64-bit và sự phân mảnh bộ nhớ do cấp phát rải rác trên heap sẽ làm tốn thêm gấp đôi dung lượng và phá hủy hoàn toàn tính liên tục của cache phần cứng.
- **Giới hạn cố hữu của cây Fenwick:** Fenwick hoàn toàn không hỗ trợ truy vấn đoạn $[l, r]$ tùy ý cho các phép toán thiếu tính nghịch đảo (như min, max, gcd) khi mảng có cập nhật điểm tùy ý. Ta không thể suy ra cực tiểu đoạn $[l, r]$ chỉ từ tiền tố $[1, r]$ và $[1, l - 1]$ vì thông tin đã bị xoá sổ.

- **Tính bất động tuyệt đối của Sparse Table:** Sparse Table đổi thời gian tiền xử lý $O(n \log n)$ và bộ nhớ $O(n \log n)$ để lấy truy vấn $O(1)$. Nhưng chỉ cần một giá trị trong mảng thay đổi, có tới $O(n)$ ô bảng bị ảnh hưởng. Việc cập nhật động trên Sparse Table là hoàn toàn bất khả thi.

Rủi ro vận hành và các lỗi im lặng nguy hiểm:

- **Xung đột thứ tự của các nhãn Lazy (Tag Composition Pitfall):** Khi bài toán có nhiều loại thao tác cập nhật đồng thời (như vừa gán đè giá trị vừa cộng dồn, hoặc vừa nhân vừa cộng), các phép toán lazy không có tính giao hoán. Nếu không xác định rõ thứ tự đại số và cập nhật đồng thời cả hai nhãn (ví dụ phép gán đè phải xoá sạch nhãn cộng trước đó), cây sẽ âm thầm trả về kết quả sai lệch hoàn toàn.
- **Quên đẩy nợ (Missing Push-Down):** Trong cây phân đoạn có lazy propagation, chỉ cần quên gọi hàm `push_down` ở đúng một nhánh rẽ (ví dụ khi đoạn truy vấn chỉ chạm một phần nút con), nút con sẽ tiếp tục mang giá trị lỗi thời. Cây vẫn chạy và trả về số, nhưng kết quả sai một cách im lặng và cực kỳ khó tái hiện trên các bộ test nhỏ.
- **Tràn số nguyên trong phép nhân độ dài đoạn:** Khi truyền nhãn lazy cộng giá trị v xuống một đoạn có độ dài len , giá trị cộng vào tổng là $v \cdot len$. Với $v \approx 10^9$ và $len \approx 10^5$, tích này vượt ngưỡng 10^{14} . Nếu viết `tree[node] += v * len` mà không ép kiểu sang `long long`, phép nhân sẽ bị tràn số 32-bit thành số âm trước khi cộng vào tổng.

CẠM BẮY

Mảng cây phân đoạn 1-indexed phải có kích thước tối thiểu $4N$, không phải $2N$. Kích thước $2^{\lceil \log_2 N \rceil + 1} - 1 < 4N$ là cận trên chặt chẽ; khai báo thiếu kích thước (ví dụ khai báo $2N$ hay $3N$) sẽ gây tràn bộ nhớ (Out-of-Bounds Write) ở các tầng lá sâu nhất, dẫn tới lỗi bộ nhớ ngầm cực kỳ khó truy vết.

1.9 Gốc rễ: từ nén dữ liệu và hình học tới bài toán truy vấn đoạn

Điều thú vị là hai cấu trúc chính của chương này ra đời ở hai ngành hoàn toàn khác nhau và không phải để giải bài “truy vấn đoạn” như ta biết ngày nay.

Cây phân đoạn xuất hiện trong *hình học tính toán* cuối thập niên 1970, gắn với tên Jon Bentley. Bài toán gốc là bài toán đâm xuyên: cho một tập đoạn thẳng trên đường thẳng, hỏi một điểm bị bao nhiêu đoạn phủ. Cấu trúc phân cấp theo đoạn ra đời từ đó, và mãi về sau cộng đồng thuật toán mới dùng nó cho bài toán truy vấn tổng/cực trị trên mảng — ứng dụng mà ngày nay hầu như là ứng dụng duy nhất người ta nhắc tới.

Fenwick ra đời năm 1994 từ một bài toán *nén dữ liệu*. Trong mã hoá số học, bộ mã cần liên tục cập nhật tần suất của từng ký hiệu và liên tục hỏi tổng tần suất lũy tích — đúng bài toán “cập nhật điểm, truy vấn tiền tố”. Peter Fenwick tìm ra cách biểu diễn bằng bit thấp nhất và công bố nó như một giải pháp cho bài toán nén; cấu trúc gọn tới mức ngày nay nó là thứ đầu tiên người ta viết ra khi cần tổng đoạn có cập nhật.

Cả hai câu chuyện minh hoạ cùng một điều đáng nhớ về cách kiến thức lan truyền: cấu trúc dữ liệu hay được sinh ra để phục vụ một ứng dụng rất hẹp, rồi được cộng đồng khác nhận ra là công cụ tổng quát. Với người học, hệ quả thực dụng là: khi bạn gặp một bài lạ, rất có thể công cụ cần dùng đã tồn tại ở một ngành khác dưới một cái tên khác — và đó cũng là lý do phần *Liên hệ ngang* có mặt trong mọi chương của quyển sách này.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Cuối thập niên 1970 — Bentley và hình học tính toán	Bài toán đâm xuyên: một điểm bị bao nhiêu đoạn phủ.	Cây phân đoạn ra đời cho hình học; ứng dụng “truy vấn tổng trên mảng” đến sau và trở thành ứng dụng chính.
1994 — Fenwick	Mã hoá số học cần cập nhật tần suất và hỏi tổng lũy tích liên tục.	Cấu trúc dựa trên bit thấp nhất; ngắn tới mức thành lựa chọn mặc định cho tổng đoạn có cập nhật.
Thập niên 1980–2000 — bài toán RMQ và LCA	Truy vấn cực tiểu trên đoạn xuất hiện khắp nơi, cần lời giải $O(1)$.	Sparse table và các lời giải tiền xử lý tuyến tính; nối RMQ với bài toán tổ tiên chung gần nhất trên cây ($C/17$).
Thập niên 2000 — cộng đồng thi đấu	Nhiều bài truy vấn không có tính kết hợp nên cây chịu.	Phân rã căn bậc hai và thuật toán Mo: đổi tính trực tuyến lấy khả năng xử lý phép toán bất kỳ.
Thập niên 2000 — cấu trúc bền vững	Cần truy vấn trạng thái ở các thời điểm quá khứ.	Cây phân đoạn bền vững bằng chia sẻ đường đi; giải được “phần tử nhỏ thứ k trong đoạn”.

1.10 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Đại số trừu tượng	Vị nhóm: phép kết hợp cộng phần tử trung hoà	Không phải chỉ tương tự — đây đúng là điều kiện toán học để cây phân đoạn hoạt động. Biết khái niệm này giúp trả lời ngay câu “bài này dùng cây phân đoạn được không”: chỉ cần kiểm tính kết hợp.
Thị giác máy tính, đồ hoạ	Kim tự tháp ảnh, mipmap, ảnh tích phân	Cùng ý tưởng tổng hợp phân cấp: mỗi tầng lưu bản tóm tắt của tầng dưới với độ phân giải giảm nửa. Khác: ở đó mục tiêu là lọc và lấy mẫu theo tỉ lệ, ở đây là trả lời truy vấn nhanh.
Cơ sở dữ liệu phân tích	Khối dữ liệu OLAP, bảng tổng hợp trước, chuỗi thời gian có tổng theo cấp	Cùng đánh đổi trả trước để truy vấn rẻ, và cùng vấn đề: dữ liệu thay đổi thì phải cập nhật tổng hợp. Cây phân đoạn chính là bản trong bộ nhớ của ý tưởng đó.
Xử lý tín hiệu	Biến đổi wavelet, phân tích đa phân giải	Cùng cấu trúc phân cấp nhị phân trên miền chỉ số. Bài học chung: biểu diễn đa mức cho phép mô tả cả thay đổi cục bộ lẫn tổng thể với chi phí lôgarit.
Tính toán song song	Cây rút gọn (reduce) và scan trên GPU	Cây phân đoạn chính là cây rút gọn được lưu lại thay vì tính xong rồi bỏ. Điều kiện dùng được cũng giống nhau: phép toán phải kết hợp — đó là lý do reduce trong mọi thư viện song song đều yêu cầu điều kiện này.
Lập trình hàm	Cấu trúc bền vững, chia sẻ cấu trúc	Cây phân đoạn bền vững là ứng dụng trực tiếp của kỹ thuật chia sẻ đường đi[3]; cùng lý do vì sao cây hợp với bất biến còn mảng thì không.
Hệ thống giám sát	Cửa sổ thời gian, tổng hợp theo cấp phút/giờ/ngày	Cùng cấu trúc phân cấp theo thời gian. Điểm giống thú vị: các hệ này cũng chỉ hỗ trợ được các phép tổng hợp có tính kết hợp, và “số giá trị phân biệt” cũng chính là phép khó ở đó — nên họ dùng cấu trúc xấp xỉ như HyperLogLog ($B/08$).

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì Fenwick tính đoạn $[l, r]$ bằng *hiệu* của hai tiền tố, nên nó cần phép toán có nghịch đảo. Với cực đại thì biết max của $[1, r]$ và của $[1, l - 1]$ không suy ra được max của $[l, r]$ — thông tin đã mất. Cây phân đoạn không dùng phép trừ mà *ghép* kết quả của các đoạn con rời nhau, nên chỉ cần tính kết hợp chứ không cần nghịch đảo. Đây là ranh giới rõ ràng: có nghịch đảo thì Fenwick (ngắn hơn, nhanh hơn); chỉ có kết hợp thì cây phân đoạn.
- A2. Vì sparse table đòi mảng *tĩnh*: cập nhật một phần tử có thể làm hỏng $O(n)$ ô (ở tầng k , tối đa 2^k ô chứa điểm đó). Xây lại toàn bảng tốn $O(n \log n)$, nhưng sửa đúng các ô bị ảnh hưởng tốn $O(n)$. Nó cũng đòi phép toán lũy đẳng, vì hai đoạn dài 2^k dùng để phủ truy vấn có phần chồng lên nhau — với tổng thì phần chồng bị đếm hai lần. Vậy sparse table thắng đúng một tình huống: mảng không đổi, phép lũy đẳng, và số truy vấn rất lớn nên $O(1)$ đáng giá.
- A3. Vì bất biến được duy trì là: giá trị tổng hợp lưu tại mỗi nút *đã đúng*, còn khoản nợ tại nút chỉ mô tả phần chưa được truyền xuống các con. Mọi thao tác muốn đọc thông tin của con đều phải đi xuống qua nút cha, và ta luôn đẩy nợ trước khi đi xuống — nên không ai từng đọc dữ liệu chưa cập nhật. Đây là lập luận bất biến chuẩn của A/04, và nó cũng chỉ ra chỗ dễ sai nhất khi cài: quên đẩy nợ ở đúng một nhánh.
- A4. Ở hằng số, không ở bậc. Fenwick là một vòng lặp trên mảng với vài phép toán bit, không đệ quy, không cấp phát, truy cập gần như tuần tự nên rất hợp cache. Cây phân đoạn thường viết đệ quy, mỗi nút truy cập hai con ở vị trí xa nhau, và dùng gấp bốn lần bộ nhớ. Với cùng $\log n$ bước, chênh lệch thực tế thường là ba tới năm lần — đủ để quyết định giữa đạt và quá thời gian.
- A5. Có, và đó là lý do chính để biết phân rã căn bậc hai. Ví dụ “có bao nhiêu giá trị phân biệt trong đoạn $[l, r]$ ” không kết hợp được: biết số giá trị phân biệt của hai nửa không cho biết số giá trị phân biệt của cả đoạn, vì không biết phần chung. Thuật toán Mo giải được bằng cách di chuyển hai đầu mút dần dần và duy trì một bảng đếm — với cái giá là mọi truy vấn phải biết trước, tức là mất tính trực tuyến.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy hình dung một dãy phố với một trăm cửa hàng, và bạn thường xuyên phải trả lời câu “tổng doanh thu từ cửa hàng số 20 tới số 57 là bao nhiêu?”. Nếu doanh thu không bao giờ đổi, bạn chỉ cần một bảng cộng dồn từ đầu phố — lấy số ở 57 trừ số ở 19 là xong.

Nhưng nếu mỗi ngày có vài cửa hàng báo lại doanh thu mới thì bảng cộng dồn hỏng: sửa một cửa hàng làm sai mọi con số phía sau nó.

Cách chữa là chia phố thành nhóm lồng nhau: hai nửa phố, mỗi nửa lại chia đôi, cứ thế xuống từng cửa hàng. Mỗi nhóm ghi sẵn tổng của nhóm mình. Khi một cửa hàng đổi số, chỉ những nhóm chứa nó — khoảng bảy nhóm với một trăm cửa hàng — phải sửa. Khi có người hỏi một khoảng, bạn ghép khoảng đó từ vài nhóm có sẵn thay vì cộng từng cửa hàng.

Còn nếu sếp nói “tăng thuế thêm 3% cho cả đoạn từ 20 tới 57” thì bạn không cần đi sửa từng cửa hàng. Bạn dán một mẫu giấy nợ lên các nhóm nằm trọn trong đoạn đó, và chỉ khi nào có ai hỏi chi tiết bên trong một nhóm thì mới truyền mẫu giấy ấy xuống các nhóm con. Đó là lazy propagation, và nó đúng vì bất kỳ ai muốn nhìn vào bên trong đều phải đi qua chỗ dán giấy.

Một điều cuối cùng đáng nhớ: cách này chỉ chạy được với những câu hỏi mà *biết đáp án của hai nửa thì suy ra được đáp án của cả đoạn*. Tổng thì được, lớn nhất thì được, nhưng “mặt hàng bán chạy nhất trong đoạn” thì không — và đó là ranh giới giữa những bài dùng được cây và những bài phải dùng cách khác.

Một câu: Chia đoạn thành các mảnh lồng nhau: sửa một chỗ chỉ đụng $\log n$ mảnh, hỏi một đoạn chỉ ghép $\log n$ mảnh.

Chỗ không rút gọn được: Việc phép toán có kết hợp hay không là tính chất của bài toán, không phải của cấu trúc — không có mẹo nào làm cho “giá trị phổ biến nhất” trở nên kết hợp được.

Thuật ngữ / câu hỏi	Trả lời
●●● Điều kiện chung của cả chương	Phép toán phải <i>kết hợp</i> : kết quả cả đoạn ghép được từ hai nửa. Cộng thêm phần tử trung hoà thì có một vị nhóm.
●●● Fenwick so với cây phân đoạn	Fenwick cần phép có <i>ngịch đảo</i> (dùng hiệu hai tiền tố), thường gọn hơn; tốc độ cần đo. Cây phân đoạn chỉ cần kết hợp, tổng quát hơn, làm được lazy.
●●● Vì sao truy vấn cây phân đoạn là $O(\log n)$	Mỗi tầng đóng góp tối đa hai nút được dùng nguyên vẹn; các nhánh còn lại hoặc nằm gọn (dùng) hoặc không giao (bỏ).
●●● Lazy propagation — bất biến	Giá trị tại nút đã đúng; khoản nợ mô tả phần chưa áp cho con. Mọi đường đi xuống đều đẩy nợ trước khi đọc con.
●●● Sparse table — điều kiện	Mảng tĩnh và phép <i>luỹ đẳng</i> ($a \oplus a = a$), vì hai đoạn phủ truy vấn chồng lên nhau. Tiền xử lý $O(n \log n)$, truy vấn $O(1)$.
●●● Bảng chọn cấu trúc	Tĩnh + có nghịch đảo: tổng tiền tố. Tĩnh + lũy đẳng: sparse table. Động + nghịch đảo: Fenwick. Động + chỉ kết hợp: cây phân đoạn. Không có trạng thái gộp gọn: xét chia khối hoặc Mo, rồi kiểm chi phí các thao tác.
●● Lỗi cài đặt phổ biến nhất	Phần tử trung hoà sai: trả 0 cho truy vấn cực đại trên đoạn rỗng khi mảng có số âm. Hãy đặt nó thành hằng số có tên.
●● Thuật toán Mo	Truy vấn ngoại tuyến, sắp theo khối của đầu trái rồi đầu phải, di chuyển hai con trỏ; $O((n + q)\sqrt{n})$. Giải được bài không kết hợp như đếm giá trị phân biệt.
●● Phân rã căn bậc hai	$\sqrt{n}$ khối, mỗi khối giữ tổng hợp; truy vấn $O(\sqrt{n})$. Cần gộp mỗi khối đủ rẻ; không tự động xử lý mọi phép toán.
●● Cây phân đoạn bền vững	Mỗi cập nhật sao chép đúng một đường $O(\log n)$ nút, chia sẻ phần còn lại; giải “phần tử nhỏ thứ k trong đoạn”.
●● Fenwick tìm vị trí theo tổng	Đi xuống theo bit để tìm vị trí nhỏ nhất có tổng tiền tố vượt x , $O(\log n)$ — thay được cây thống kê thứ tự ở B/09.
● Hai chiều	Fenwick 2D cho tổng trên hình chữ nhật; cây phân đoạn của cây phân đoạn cho $O(\log^2 n)$ với phép tổng quát hơn, tốn bộ nhớ.
●● Phép biến đổi Bentley–Saxe	Bài toán phân rã được cộng một bộ cấu trúc tính kích thước 1, 2, 4, ..., hợp nhất như phép cộng nhị phân: thêm được phần tử trực tuyến với giá một thừa số $O(\log n)$. Không hỗ trợ xoá.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên học Fenwick hay cây phân đoạn trước?

| Fenwick trước, vì nó ngắn, cho kết quả nhanh và dạy đúng ý tưởng cốt lõi “mỗi phần tử nằm trong $\log n$ đoạn tóm tắt”. Nhưng đừng dừng ở đó: cây phân đoạn mới là cấu trúc tổng quát, và một khi đã quen thì bạn viết nó gần như tự động. Lộ trình hợp lý là Fenwick cho tổng, rồi cây phân đoạn không lazy, rồi lazy, rồi mới tới các mở rộng.

2. Cây phân đoạn nên cài đệ quy hay lặp?

| Bản đệ quy dài hơn nhưng dễ mở rộng và bắt buộc nếu cần lazy; bản lặp (cây phân đoạn không đệ quy) ngắn hơn và nhanh hơn nhưng khó thêm lazy và khó xử lý các phép không giao hoán. Lời khuyên thực dụng: học kỹ bản đệ quy trước vì nó là bản dùng được cho mọi bài, rồi mới thêm bản lặp vào thư viện riêng cho những bài chỉ cần cập nhật điểm.

3. Bài yêu cầu “gán mọi phần tử trong đoạn bằng x ” thay vì cộng — lazy có khác gì không?

| Có một khác biệt quan trọng: phép gán *không giao hoán* với phép cộng, nên khi hai loại cập nhật cùng xuất hiện, bạn phải định nghĩa rõ cách gộp hai khoản nợ và thứ tự áp dụng. Quy tắc an toàn: biểu diễn khoản nợ dưới dạng một phép biến đổi affine ($a \mapsto ka + b$) và định nghĩa phép hợp thành của hai biến đổi. Khi đó mọi thứ trở lại đúng khuôn “vị nhóm”, chỉ là vị nhóm của các phép biến đổi thay vì của các giá trị.

4. Khi nào tôi nên dùng thuật toán Mo thay vì cố tìm cấu trúc dữ liệu?

| Khi cả ba điều kiện cùng đúng: mọi truy vấn biết trước; thao tác thêm/bớt một phần tử ở biên là $O(1)$; và $O((n+q)\sqrt{n})$ nằm trong ngân sách. Điều kiện thứ hai hay bị bỏ qua — nếu mỗi bước di chuyển con trỏ tốn $O(\log n)$ thì tổng thành $O((n+q)\sqrt{n} \log n)$ và thường quá giờ. Trước khi dùng Mo, hãy luôn tự hỏi “phép này có kết hợp không” một lần nữa, vì cây phân đoạn luôn tốt hơn khi dùng được.

5. Nén toạ độ là gì và vì sao nó hay đi kèm chương này?

| Là việc ánh xạ các giá trị thật — có thể tới 10^9 — về các chỉ số liên tiếp $0, 1, 2, \dots$ bằng cách sắp xếp và loại trùng. Nó cần thiết vì Fenwick và cây phân đoạn cấp phát bộ nhớ theo kích thước miền giá trị, chứ không theo số phần tử. Với 10^5 giá trị nằm rải rác trong $[0, 10^9]$, nén xuống còn 10^5 chỉ số làm cấu trúc khả thi. Đây là bước tiền xử lý gần như bắt buộc cho các bài đếm nghịch thế hay truy vấn theo giá trị.

6. Có cách nào cập nhật đoạn và truy vấn đoạn chỉ bằng Fenwick không?

| Có, dùng hai mảng Fenwick. Ý tưởng: viết tổng tiền tố sau các cập nhật đoạn dưới dạng một biểu thức tuyến tính theo chỉ số, rồi tách thành phần hệ số và phần hằng, mỗi phần một Fenwick. Kết quả là code vẫn ngắn hơn cây phân đoạn có lazy và nhanh hơn đáng kể — đáng để có sẵn trong thư viện riêng. Hạn chế: chỉ áp dụng cho tổng, không mở rộng sang cực trị.

7. Truy vấn cực tiểu trên đoạn có liên quan gì tới bài toán tổ tiên chung gần nhất trên cây?

| Hai bài quy dẫn được về nhau và đây là một liên hệ đẹp. Tổ tiên chung gần nhất của hai đỉnh chính là đỉnh có độ sâu nhỏ nhất trong đoạn tương ứng của dãy Euler khi duyệt cây — tức là một truy vấn cực tiểu trên đoạn. Ngược lại, mọi bài cực tiểu trên đoạn cũng dựng được thành bài tổ tiên chung trên cây Cartesian của mảng. Nhờ quy dẫn đó, một lời giải tốt cho bài này lập tức là lời giải tốt cho bài kia — chi tiết ở C/17.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Tổng đoạn có cập nhật điểm	CSES “Dynamic Range Sum Queries”[4]	Fenwick; viết lại từ trí nhớ tới khi không cần tra.
● Cực tiểu đoạn với mảng tĩnh	CSES “Static Range Minimum Queries”[4]	Sparse table; giải thích vì sao cách này sai nếu đổi sang tổng.
●● Cực tiểu đoạn có cập nhật điểm	CSES “Dynamic Range Minimum Queries”[4]	Cây phân đoạn; chú ý phần tử trung hoà.
●● Đếm số nghịch thế bằng Fenwick sau khi nén toạ độ	Bài kinh điển	Hai kỹ thuật ghép lại; so với cách sắp xếp trộn ở C/14.
●● Cộng một giá trị vào đoạn, hỏi giá trị tại một điểm	CSES “Range Update Queries”[4]	Fenwick trên mảng hiệu; đối chiếu với cách mảng hiệu thuần ở B/06.
●● Cộng vào đoạn và hỏi tổng đoạn	Bài kinh điển	Cây phân đoạn có lazy; hoặc hai Fenwick — hãy cài cả hai và so thời gian.
●●● Đoạn con có tổng lớn nhất, có cập nhật điểm	Bài thi đấu kinh điển	Mỗi nút lưu bốn giá trị (tổng, tiền tố tốt nhất, hậu tố tốt nhất, đáp án); ví dụ đẹp nhất về việc thiết kế phép gộp.
●●● Số giá trị phân biệt trong đoạn	Bài kinh điển	Thuật toán Mo hoặc cách ngoại tuyến với Fenwick; giải thích vì sao cây phân đoạn không dùng được trực tiếp.
●●● Phần tử nhỏ thứ k trong đoạn	Bài thi đấu kinh điển	Cây phân đoạn bền vững; hiệu của hai phiên bản chính là “đếm trong đoạn”.
●●● Tổng trên hình chữ nhật có cập nhật điểm	CSES “Forest Queries II”[4]	Fenwick hai chiều; so bộ nhớ với cây phân đoạn hai chiều.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●●● Cài phân rã căn bậc hai cho một truy vấn không kết hợp tự chọn	Bài tập thiết kế	Mục tiêu là tự nhận ra ranh giới “kết hợp được hay không”, không phải cài cho xong.

Kết chương và đọc tiếp

Chương này khép lại câu chuyện bắt đầu từ B/06: từ tổng tiền tố tĩnh, qua Fenwick và cây phân đoạn cho dữ liệu động, tới phân rã căn bậc hai cho những phép toán mà cây chịu. Quyết định luôn chỉ dựa trên hai câu hỏi — phép toán có kết hợp không, và mảng có thay đổi không — và bảng ở mục 4 là thứ đáng thuộc nhất.

Chương tiếp theo là cấu trúc có tỉ lệ sức mạnh trên độ dài code cao nhất trong cả quyển sách: hợp nhất tập hợp, chỉ hai chục dòng nhưng đứng sau thuật toán cây khung nhỏ nhất, các bài liên thông ngoại tuyến, và một phân tích khấu hao đẹp tới mức hàm chi phí của nó tăng chậm hơn mọi hàm bạn từng gặp (B/11).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Bentley & Saxe, “Decomposable Searching Problems I: Static-to-Dynamic Transformation”	Journal of Algorithms, 1980	Nguồn của phép biến đổi ở mục 6. Đọc phần định nghĩa bài toán phân rã được và bảng các phép biến đổi; phần tối ưu tổ hợp đọc sau.
nền tảng	Fenwick, “A New Data Structure for Cumulative Frequency Tables”	Software: Practice and Experience, 1994	Bài gốc của cây chỉ số nhị phân, viết cho bài toán nén số học. Mười trang, và cách trình bày lowbit ở đây rõ hơn phần lớn tài liệu về sau.
tra khi cần	Bentley, “Programming Pearls”	Addison-Wesley, tái bản 2000	Đọc các chương về tổng tiền tố và về thiết kế cấu trúc dữ liệu nhỏ; văn phong ngắn, hợp đọc rải rác.
tra khi cần	Demaine, khoá học MIT 6.851 <i>Advanced Data Structures</i>	tài liệu khoá học công khai, MIT	Bài giảng về cấu trúc bền vững và về “động hoá” cấu trúc tĩnh, đi tiếp đúng hướng của mục này.

Nguồn

- [1] J. L. Bentley and J. B. Saxe, “Decomposable Searching Problems I: Static-to-Dynamic Transformation,” *Journal of Algorithms*, vol. 1, no. 4, pp. 301–358, 1980.
- [2] AtCoder. “AC Library: Lazy Segtree.” (), address: https://atcoder.github.io/ac-library/master/document_en/lazysegtree.html (visited on 09/07/2026).
- [3] C. Okasaki, *Purely Functional Data Structures*. Cambridge University Press, 1998.
- [4] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).